

C BOOLEANS

LOGICAL TRUTH & COMPUTATIONAL
DECISIONS



Essential 10-Page Technical Reference

01. Concept of Truth

In computer science, a Boolean is a data type that has one of two possible values: **True** or **False**. It is the fundamental basis of all binary logic and decision-making in programming.

***The C Paradox:** Originally (in K&R C), the C language did not have a dedicated 'boolean' data type. It relied entirely on numeric representation.*

This minimalist approach meant that logic was handled using standard integers.

02. The 0 and 1 Rule

In classical C, logical truth is determined by integer values:

- **0** represents **False**.
- **1** (or any non-zero value like -5, 10, 100) represents **True**.

```
int isTrue = 1;
if (isTrue) {
    printf("This will run!");
}
```

03. Boolean in Control Flow

Control structures like `if`, `while`, and `for` implicitly treat the expression inside their parentheses as a boolean test.

```
if (5 > 2) { // Evaluate to 1 (True)
  // Execute code
}

if (0) {
  // This block is unreachable code
}
```

04. The stdbool.h Header

With the introduction of the **C99 Standard**, C finally added a official boolean type via the `stdbool.h` library.

```
#include <stdbool.h>
bool isReady = true;
```

The New Keywords:

- `bool`: The data type name.
- `true`: Constant representing 1.
- `false`: Constant representing 0.

05. Size of Bool

Even though a boolean only needs 1 bit of information (0 or 1), the computer cannot address a single bit directly in memory.

Size: Typically, a `bool` occupies **1 Byte** (8 bits) of memory.

Using `sizeof(bool)` will usually return 1 on most modern compilers.

06. Relational Output

Relational operators (`==`, `!=`, `>`, `<`) automatically return boolean results that can be stored or printed as integers.

```
int a = 10, b = 20;  
bool result = (a < b); // result = true (1)
```

Printing `true` or `false` using `printf` will output 1 or 0 respectively using the `%d` specifier.

07. Handling Multiple Truths

Booleans are most powerful when combined with Logical Operators:

- **AND (&&):** True only if both are true.
- **OR (||):** True if at least one is true.
- **NOT (!):** Flips the value.

```
bool canVote = (hasID && isAdult);
```

08. Pitfalls & Errors

- **Mismatched Header:** Trying to use `bool` without `#include <stdbool.h>`.
- **Integer confusion:** Forgetting that `-1` is considered **True**.
- **String confusion:** Booleans are not strings. `bool b = "true";` is an error.

09. Summary & Best Practices

- 0 = False, Non-zero = True.
- Use `stdbool.h` for modern, readable code.
- Booleans make conditions like `if` and `while` possible.
- Names should be descriptive: `isActive`, `hasError` instead of just `flag`.

END OF C BOOLEANS GUIDE